

Cours Atelier de Programmation II

Langage C

Faculté des Sciences de Tunis

Année Universitaire 2025/2026

Chapitre 7 : Arbres

- ❖ **Arbres**
- ❖ **Arbres Binaires**
- ❖ **Arbres Binaires de Recherche - ABR**

Cours : les arbres en langage C

Structure hiérarchique • arbres binaires • arbres binaires de recherche • parcours
• implémentation en C

Pourquoi étudier les arbres ?

Les arbres modélisent les hiérarchies, les dossiers, les expressions, les index et de nombreuses structures de recherche efficaces. En C, ils introduisent les pointeurs, la récursivité et la gestion mémoire dynamique.

Mots-clés

Racine • parent • enfant • feuille • profondeur (depth)
• hauteur (height)

Structures modélisées

Dossiers, expressions, menus, index, arbres de décision, grammaires.

Objectif du cours

Comprendre la logique des arbres puis savoir les implémenter proprement en C.

Plan du cours

Progression sur 20 slides

Partie 1 — Concepts fondamentaux

- Définition d'un arbre
- Vocabulaire essentiel
- Types d'arbres
- Représentation mémoire en C

Partie 2 — Parcours et algorithmes

- Parcours préfixe, infixe, postfixe
- Parcours en largeur (BFS)
- Fonctions récursives
- Analyse de complexité

Partie 3 — ABR et applications

- Arbre binaire de recherche (ABR)
- Insertion, recherche,
- Cas pratiques
- Exercices de synthèse

1. Définition et vocabulaire d'un arbre

- Un **arbre** est une structure de données **hiérarchique** composée de **nœuds** reliés par des **arêtes**.
- Le nœud tout en haut est la **racine**; chaque nœud peut avoir zéro, un ou plusieurs enfants.
- Un nœud sans enfant est une **feuille**.
- La **profondeur (depth)** d'un nœud est le **nombre d'arêtes** depuis ce **nœud** vers la **racine**;
- La **hauteur (height)** d'un arbre est la **profondeur maximale** : le nombre d'arêtes entre un **nœud** vers la **feuille la plus éloignée**.
- Dans un arbre, il n'existe qu'un *seul chemin* entre la *racine* et un *nœud donné*.

Racine

Premier nœud, point d'entrée de tout l'arbre.

Parent / enfant

Relation hiérarchique entre un nœud et ses descendants directs.

Feuille / hauteur

Une feuille n'a aucun enfant;
La **hauteur** mesure la profondeur maximale dans l'arbre = **Distance** (Racine, la Feuille la plus éloignée).

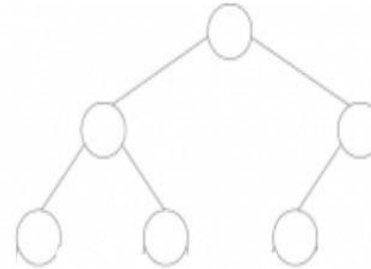
2. Types d'arbres courants

Arbre général

Un nœud peut avoir un nombre quelconque d'enfants.
Exemple : arborescence de dossiers.

Arbre binaire complet

Tous les niveaux sont remplis, sauf éventuellement le dernier, rempli de gauche à droite.

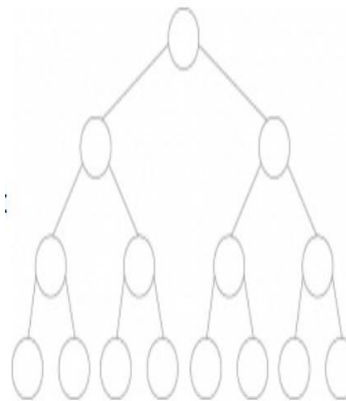


Arbre binaire

Chaque nœud possède au plus deux enfants : gauche et droit.

Arbre binaire parfait

Tous les nœuds internes ont exactement deux enfants et toutes les feuilles sont au même niveau.



Arbres spécialisés

- ABR : permet une recherche ordonnée.
- Arbre équilibré : hauteur des ss-ArbreG et ss-ArbreD est presque la même
- Trie : un arbre utilisé pour stocker des mots/préfixes.

En langage C, on commence généralement par

- les arbres binaires (= structure de base),
- puis l'ABR (= cas particulier avec règles).

3. Représentation mémoire en langage C

```
typedef struct Noeud {  
    int valeur;  
    struct Noeud *gauche;  
    struct Noeud *droite;  
} Noeud;
```

Interprétation

Chaque nœud stocke :

- 1) une donnée (ici un entier)
- 2) l'adresse du *fils gauche*
- 3) l'adresse du *fils droit*

Un pointeur **NULL** signifie *l'absence d'enfant*.

Important : l'arbre est manipulé à travers un pointeur vers la racine. Toute création de nœud se fait en mémoire dynamique avec *malloc*.

4. Création d'un nœud et construction d'un petit arbre

```
1 Noeud *creerNoeud (int val) {
2   Noeud *nv =
3   (Noeud*) malloc (sizeof(Noeud));
4   if (nv == NULL) return NULL;
5   nv->valeur = val;
6   nv->gauche = NULL;
7   nv->droite = NULL;
8   return nv;
9 }
9 //Main
10 Noeud *racine = creerNoeud(10);
11 racine->gauche = creerNoeud(5);
12 racine->droite = creerNoeud(20);
```

Étape 1

Allouer un nœud avec `malloc` puis vérifier que le pointeur n'est pas NULL (BP).

Étape 2

Initialiser la valeur puis mettre
gauche = NULL et
droite = NULL.

Étape 3

Relier les nœuds en affectant les pointeurs
racine->gauche,
racine->droite,
etc.

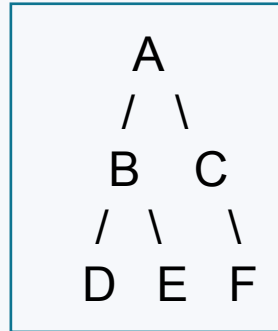
5. Parcours en profondeur (DFS- Depth First Search) : Préfixe (preorder) **R|G|D**

Principe

Ordre de visite :

- 1) **racine**
- 2) sous-arbre **gauche**
- 3) sous-arbre **droit**

```
1 void prefixe (Noeud *r) {  
2     if (r != NULL) {  
3         printf("%d ", r->valeur);  
4         prefixe (r->gauche);  
5         prefixe (r->droite);  
6     }  
7 }
```



Mnémonique

Préfixe = on traite la racine avant ses enfants.

Exemple d'ordre

A → B → D → E → C → F

Racine → Gauche → Droite

Cas d'usage

Copie d'arbre, sauvegarde, affichage hiérarchique.

6. Parcours en profondeur (DFS-Depth First Search) :

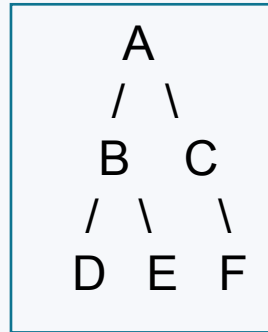
Infixe (inorder) G|R|D

Principe

Ordre de visite :

- 1) sous-arbre **g**auche
- 2) **racine**
- 3) sous-arbre **d**roit

```
1 void infixe (Noeud *r) {  
2   if (r != NULL) {  
3     infixe (r->gauche);  
4     printf("%d ", r->valeur);  
5     infixe (r->droite);  
6   }  
7 }
```



Mnémonique

Infixe = **g**auche, puis **racine**, puis **d**roite.

Exemple d'ordre

D → B → E → A → C → F

Point important

Dans un ABR, ce parcours affiche les valeurs dans l'ordre croissant.

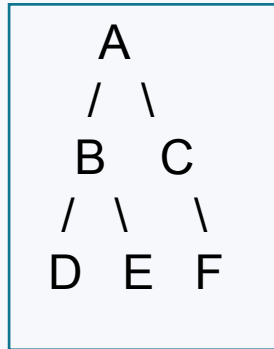
7. Parcours en profondeur (DFS-Depth First Search) : Postfixe (postorder) G|D|R

Principe

Ordre de visite :

- 1) sous-arbre **g**auche
- 2) sous-arbre **d**roit
- 3) **r**acine

```
1 void postfixe (Noeud *r) {  
2     if (r != NULL) {  
3         postfixe (r->gauche);  
4         postfixe (r->droite);  
5         printf("%d ", r->valeur);  
6     }  
7 }
```



Mnémonique

Postfixe = on traite la **racine** en **dernier**.

Exemple d'ordre

D → E → B → F → C → A

Cas d'usage

Suppression/Libération mémoire
complète ou évaluation d'arbres
d'expression.

8. Parcours en largeur (level-order/ BFS)

• Breadth – First – Search

- Le parcours en **largeur** visite les nœuds niveau par niveau, de gauche à droite.
- Il nécessite une file (queue) pour mémoriser les nœuds découverts mais non encore traités.
- Principe : enfiler la racine, puis répéter : défiler un nœud, l'afficher, enfiler ses enfants.

```
void parcoursLargeur(Noeud *racine) {  
    if (racine == NULL) return;  
    File f;  
    initFile(&f);  
    enfiler(&f, racine);  
    while (!fileVide(&f)) {  
        Noeud *x = defiler(&f);  
        printf("%d ", x->valeur); // traiter(x)  
        if (x->gauche != NULL)  
            enfiler(&f, x->gauche);  
        if (x->droite != NULL)  
            enfiler(&f, x->droite);  
    }  
}
```

Principe

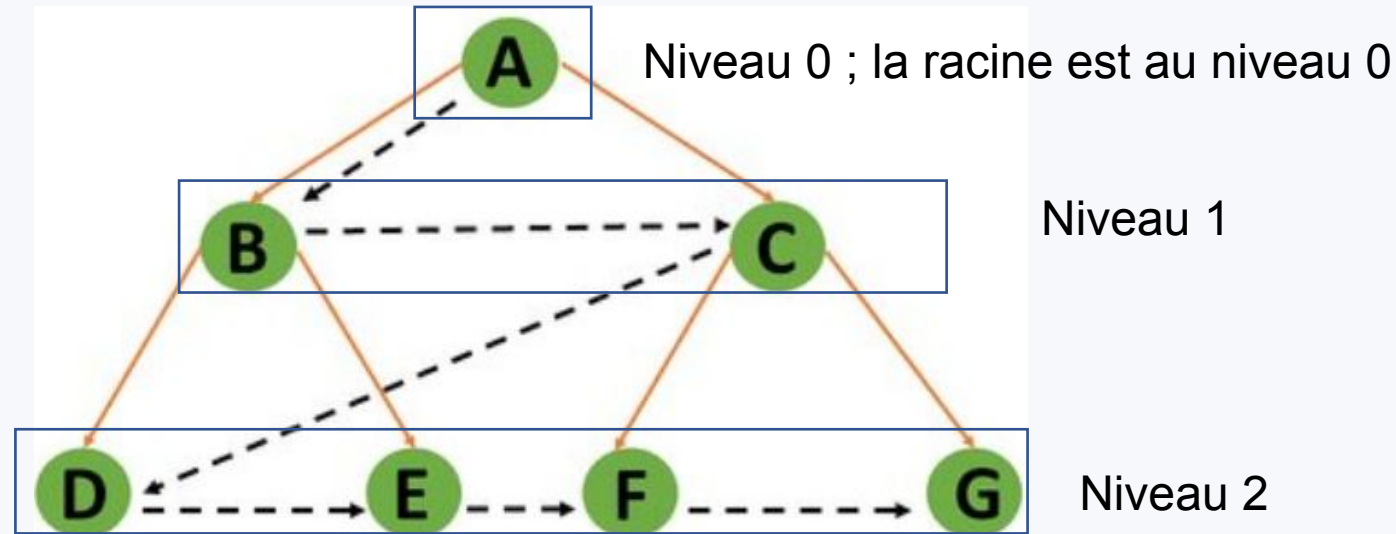
Utiliser une file pour visiter les nœuds niveau par niveau.

Exemple d'ordre

0 → 1 → 2 → 3 → 4 → 5 → 6 → 7

Quand l'utiliser ?

Affichage par niveaux, calculs couche par couche, analyse de largeur.



9. Fonctions récursives utiles sur un arbre

```
// le nombre total de nœuds (Nodes)
1 int taille (Noeud *r) {
2     if (r == NULL) return 0;
3     return 1 + taille (r->gauche) +
      taille (r->droite);
4 }
5 //le nombre total d'arêtes(Edges)
6 int hauteur (Noeud *r) {
7     if (r == NULL) return -1;
8     // hauteur en nombre d'arêtes
9     int hg = hauteur (r->gauche);
10    int hd = hauteur (r->droite);
11    return 1 + (hg > hd ? hg : hd);
12 }
```

Idée de la récursivité

Chaque fonction traite la racine puis délègue le traitement des sous-arbres à elle-même. L'arbre se décompose naturellement en sous-problèmes.

Conseil pratique

Toujours définir clairement le cas de base : souvent `r == NULL`. C'est lui qui évite la récursion infinie.

10. Arbre binaire de recherche (ABR)

- Dans un ABR, pour chaque nœud x : toutes les valeurs du sous-arbre **gauche** sont **strictement inférieures** à x .
- Toutes les valeurs du sous-arbre **droit** sont **strictement supérieures** à x .
- Cette propriété permet d'effectuer la recherche, l'insertion et souvent la suppression plus efficacement qu'avec une liste non triée.

Propriété-clé

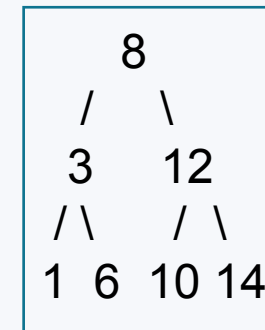
Valeurs plus petites à gauche, plus grandes à droite.

Conséquence

À chaque comparaison, on choisit un seul sous-arbre à explorer.

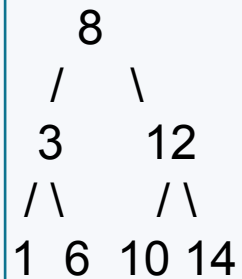
Parcours infixe (G|R|D)

Affiche les valeurs triées : 1 3 6 8 10 12 14



11. Recherche dans un ABR

```
1 Noeud *rechercher (Noeud *r, int val) {  
    //Cas de base (pas trouvé || trouvé)  
2     if (r == NULL || r->valeur == val)  
3         return r;  
4  
5     if (val < r->valeur)  
6         return rechercher (r->gauche, val);  
7     else  
8         return rechercher (r->droite, val);  
9 }
```



Idée centrale

À chaque comparaison, on élimine tout un sous-arbre qui ne peut pas contenir la valeur cherchée.

Exemple

Pour chercher 10 : $10 > 8$ donc aller à droite; $10 < 12$ donc aller à gauche; trouvé.

Bon cas : arbre équilibré $\Rightarrow$ recherche rapide.

Recherche par élimination de branche $= O(\log(n))$

12. Insertion dans un ABR

```
1 Noeud *insérer (Noeud *r, int val) {  
    //Cas de base  
2     if (r == NULL)  
3         return créerNoeud(val);  
4     if (val < r->valeur) // Comparaison avec la racine  
        //on va à gauche  
    r->gauche = insérer (r->gauche, val);  
5     else if (val > r->valeur)  
        //on va à droite  
6         r->droite = insérer (r->droite, val);  
7  
8     return r; //doublons ignorés ici (on ne fait rien)  
9 }
```

Étapes logiques

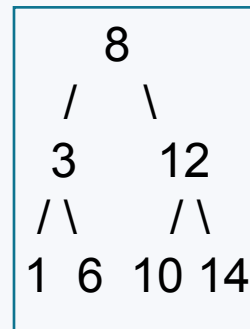
Comparer la nouvelle valeur à la racine, descendre à gauche ou à droite, puis créer un nœud dès qu'on atteint NULL.

Exemple

Insertion de 7 dans l'ABR {8,3,12,1,6,10,14} : $7 > 3$, $7 > 6$, donc 7 devient fils droit de 6.

Complexité

$O(h)$, où h est la hauteur de l'arbre.



Insertion de 7 dans l'ABR

Inserer(8, 7); $7 < 8$ -> aller à gauche

Inserer(3, 7); $7 > 3$ -> aller à droite

Inserer(6, 7); $7 > 6$ -> aller à droite

Inserer(NULL, 7); -> cas de base

-> créerNoeud 7

13. Complexité des opérations

Opération	Arbre binaire quelconque	ABR équilibré	ABR dégénéré
Recherche	$O(n)$	$O(\log n)$	$O(n)$
Insertion	selon stratégie	$O(\log n)$	$O(n)$

Interprétation

Un ABR n'est performant que si sa hauteur reste raisonnable.

Lorsqu'il se dégrade en chaîne (comme une liste chaînée), il perd l'avantage logarithmique.

$O(\log n)$: très peu d'opérations, exécution beaucoup plus rapide.

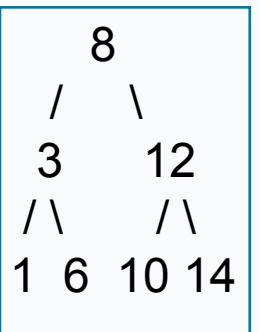
14. Programme complet minimal : ABR en C

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Noeud { int valeur;
    struct Noeud *gauche, *droite;
} Noeud;

Noeud *creerNoeud( int v) {
    Noeud *nv = malloc(sizeof(Noeud));
    if (!nv) return NULL;
    nv->valeur=v;
    nv->gauche=nv->droite=NULL;
    return nv; }
Noeud *insérer (Noeud *r, int v) {
    if(!r) return creerNoeud(v);
    if(v<r->valeur)
        r->gauche= insérer (r->gauche,v);
    else
        if(v>r->valeur)
            r->droite= insérer (r->droite,v);
        return r; }
```

```
void infixe (Noeud *r) {
    if(r){
        infixe (r->gauche);
        printf ("%d ", r->valeur);
        infixe (r->droite);
    }
}

int main(void) { int
t[]={8,3,12,1,6,10,14};
    Noeud *racine = NULL;
    for (int i=0; i<7; i++)
        racine = insérer (racine, t[i]);
    infixe (racine);
    // affiche : 1 3 6 8 10 12 14
    return 0;
}
```



15. Gestion mémoire, erreurs fréquentes et bonnes pratiques

Erreurs fréquentes

- Oublier de tester malloc
- Déréférencer un pointeur NULL
- Perdre l'adresse de la racine après insertion/suppression
- Oublier free lors de la suppression
- Confondre hauteur (qui est le nombre d'arêtes) et le nombre de nœuds (taille)

Bonnes pratiques

- Initialiser les pointeurs à NULL
- Écrire de petites fonctions bien nommées
- Tester les cas limites : arbre vide, un seul nœud, feuille
- Tracer l'arbre à la main pour vérifier les algorithmes

Libération complète

Pour libérer tout l'arbre, utiliser un parcours postfixe :

```
liberer(gauche);  
liberer(droite);  
free(r);
```

Ainsi, on libère d'abord les enfants puis le parent.

16. Applications : arbres d'expression et tas

Arbre d'expression

Chaque noeud interne est un opérateur et chaque feuille un opérande.

Exemple pour $(a + b) * c$:

- racine = *
- fils gauche = +
- fils droit = c

Le parcours postfixe (G|D|R) est souvent utile pour l'évaluation.

Exemple

$(a + b) * c$ se lit naturellement comme un arbre.

```
  *  
 / \  
+  c  
/ \  
a  b
```

Noeuds internes

Opérateurs : +, -, *, / ...

Feuilles

Variables ou constantes : a, b, c, 5, 12 ...

Tas (heap)

Arbre binaire complet utilisé pour les files de priorité et le tri par tas.